

Supplemental Material to “ForLion: An R Package for Finding Optimal Experimental Designs with Mixed Factors”

by Siting Lin, Yifei Huang, and Jie Yang

S1 Function arguments: We provide quick reference tables for the main functions in the [ForLion](#) package;

S2 Assessing the sensitivity of the locally D-optimal design against random seeds: We rerun the algorithm for the example in Section 4.1 with 10 randomly generated random seeds to assess the sensitivity of the obtained locally D-optimal design;

S3 Another example: A model with a three-level discrete factor and an interaction involving the continuous factor: We consider a different model for the ESD experiment in Section 4.2 involving a three-level discrete factor and an interaction term involving the continuous factor.

S1 Function arguments

In this section, we provide a quick reference for the arguments used in the main functions of the [ForLion](#) package. Table [S1](#) lists the specialized arguments for `ForLion_MLM_Optimal` and `ForLion_GLM_Optimal`, which construct locally D-optimal approximate designs. Table [S2](#) summarizes the additional arguments for `EW_ForLion_MLM_Optimal` and `EW_ForLion_GLM_Optimal`, which construct EW D-optimal approximate designs under unknown parameter values. Table [S3](#) collects the arguments commonly used by the main functions.

Table S1: Special arguments for two [ForLion](#) functions

Function	Argument	Description
<code>ForLion_MLM_Optimal</code>	<code>J</code>	Number of response categories in an MLM.
	<code>bvec</code>	Vector of assumed parameter values.
	<code>Fi.func</code>	Function to calculate row-wise Fisher information F_i , default <code>Fi_MLM_func</code> .
	<code>optim_grad</code>	TRUE or FALSE, default FALSE, whether to use analytical or numerical gradient function when searching for a new design point.
<code>ForLion_GLM_Optimal</code>	<code>logscale</code>	TRUE or FALSE, whether or not to run the lift-one step in log-scale, i.e., using <code>liftoneOptimal_log_GLM_func()</code> or <code>liftoneOptimal_GLM_func()</code> .

Table S2: Special arguments for two [EW_ForLion](#) functions

Function	Argument	Description
<code>EW_ForLion_MLM_Optimal</code>	<code>bvec_matrix</code>	Matrix of bootstrapped or simulated parameter values.
	<code>EW_Fi.func</code>	Function to calculate entry-wise expectation of Fisher information F_i , default <code>EW_Fi_MLM_func</code> .

	optim_grad	TRUE or FALSE, default FALSE, whether to use analytical or numerical gradient function when searching for a new design point.
EW_ForLion_GLM_Optimal	joint_Func_b	Prior distribution function of model parameters
	Integral_based	TRUE or FALSE, whether or not integral-based EW D-optimality is used, FALSE indicates sample-based EW D-optimality is used.
	b_matrix	Matrix of bootstrapped or simulated parameter values.
	Lowerbounds	Vector of lower ends of ranges of prior distribution for model parameters.
	Upperbounds	Vector of upper ends of ranges of prior distribution for model parameters.
	logscale	TRUE or FALSE, whether or not to run the lift-one step in log-scale, i.e., using <code>EW_liftoneDoptimal_log_GLM_func()</code> or <code>EW_liftoneDoptimal_GLM_func()</code> .

Table S3: Common arguments in four key functions

Argument	Description
xlist_fix	List of discrete-factor experimental settings under consideration, default NULL indicating a list of all possible discrete-factor experimental settings will be used.
h.prime	User-supplied derivative function for continuous factors $\mathbf{x}_{(1)} \in \mathbb{R}^k$. For MLMs: return a list of $J \times p$ matrices $\partial \mathbf{X}_{\mathbf{x}} / \partial x_j$, $j = 1, \dots, k$ (numerical derivatives if omitted). For GLMs: return the $p \times k$ matrix $\partial \mathbf{h}(\mathbf{x}) / \partial \mathbf{x}_{(1)}$ (if omitted, support only main effects or interactions among discrete factors).
n.factor	Vector of numbers of distinct levels, “0” indicating continuous factors that always come first, “2” or more for discrete factors, and “1” not allowed.
factor.level	List of distinct factor levels, “(min, max)” for continuous factors that always come first, finite sets for discrete factors.
var_names	Names for the design factors, Must have the same length as factor.level ; default “X1”, “X2”, “X3”, etc.
hfunc	Function for generating the corresponding model matrix or predictor vector, given an experimental setting or design point.
link	Link function, for GLMs including “logit” (default), “probit”, “cloglog”, “loglog”, “cauchit”, “log”, and “identity” (LM); for MLMs including “continuation” (default), “baseline”, “cumulative”, and “adjacent”.
delta0	Merging threshold for initial design, such that, $\ \mathbf{x}_i^{(0)} - \mathbf{x}_j^{(0)}\ \geq \delta_0$ for $i \neq j$, default 10^{-5} .
epsilon	Numerical tolerance. A nonnegative number is regarded as numerical zero if less than epsilon, default 10^{-12} .
reltol	Relative tolerance as converging threshold, default 10^{-5} .
delta	Relative difference as merging threshold for the merging step, two points satisfying $\ \mathbf{x}_i^{(t)} - \mathbf{x}_j^{(t)}\ < \delta$ may be merged, default 0, can be different from delta0 for the initial design.
maxit	Maximum number of iterations allowed, default 100.
random	TRUE or FALSE, whether or not to repeat the lift-one step multiple times with random initial allocations, default FALSE.
nram	Number of times repeating the lift-one step with random initial allocations, valid only if random is TRUE, default 3.
random.initial	TRUE or FALSE, whether or not to repeat the whole procedure multiple times with random initial designs, default FALSE.

<code>nram.initial</code>	Number of times repeating the whole procedure with random initial designs, valid only if <code>random.initial</code> is TRUE, default 3.
<code>rowmax</code>	Maximum number of points in initial designs, default NULL indicating no restriction.
<code>Xini</code>	Initial list of design points, default NULL indicating automatically generating an initial list of design points.
<code>pini</code>	A numeric vector specifying the initial weights for the design points in <code>Xini</code> , default NULL indicating automatically generating an uniform weights of design points.

S2 Assessing the sensitivity of the locally D-optimal design against random seeds

To assess the seed-to-seed variability of locally D-optimal designs for the example in Section 4.1, we rerun the algorithm with 10 randomly generated random seeds (733, 633, 993, 294, 557, 623, 859, 108, 164 and 176), while keeping all other settings fixed.

For each run, we record the number of support points, the minimum distance among support points, and the relative D-efficiency with respect to the reference design obtained using `set.seed(123)` under the same algorithm settings.

Figure S1 summarizes the results. Across all runs, the relative efficiencies range from 99.99963% to 100.0001%, which are essentially 100% up to the specified numerical tolerance. The number of support points vary slightly across runs. These results indicate that different random seeds may lead to slightly different support points or weights, but the resulting D-efficiencies are practically the same in this example.

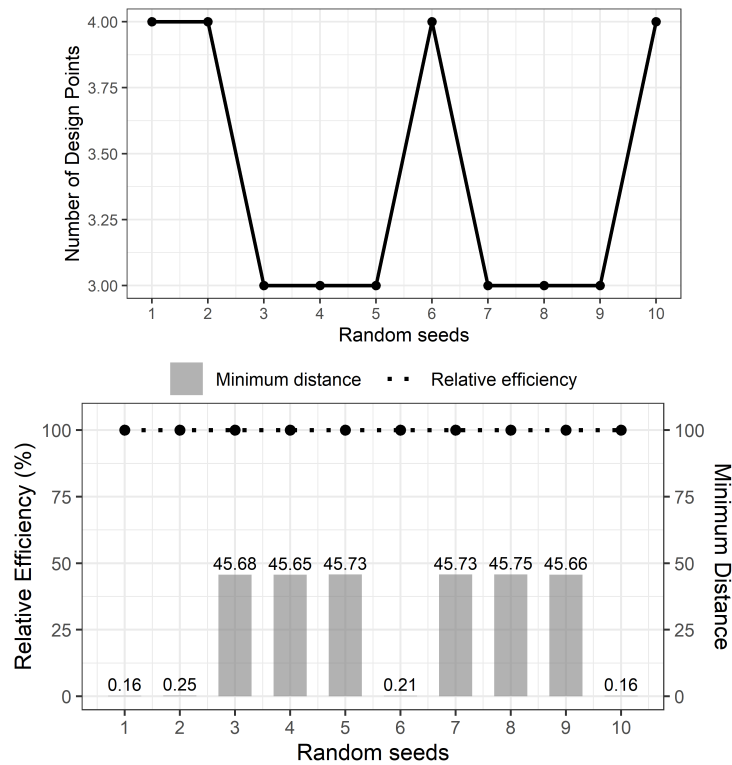


Figure S1: Number of design points, minimum distance among design points (rounded to two decimals), and relative efficiency w.r.t. the design with $\delta = 0.15$ and random seed 123

S3 Another example: A model with a three-level discrete factor and an interaction involving the continuous factor

For illustration purposes, we consider a different model for the ESD experiment in Section 4.2 by treating **Pulse** as a three-level qualitative factor with levels $\{-1, 0, 1\}$ and adding an interaction between the continuous factor **Voltage** and **Pulse**. We use this example to demonstrate how a qualitative factor and its interaction with a continuous factor are implemented through `factor.level`, `hfunc`, and `h.prime`.

We retain the notation **LotA** (x_1), **LotB** (x_2), **ESD** (x_3), **Pulse** (x_4), and **Voltage** (x_5), where $x_5 \in [25, 45]$. We take $x_4 = 1$ as the reference level and define two dummy variables, namely

$$\text{Pul1} = \begin{cases} 1, & \text{if } x_4 = -1, \\ 0, & \text{otherwise;} \end{cases} \quad \text{and} \quad \text{Pul2} = \begin{cases} 1, & \text{if } x_4 = 0, \\ 0, & \text{otherwise.} \end{cases}$$

Then the three levels $-1, 0, 1$ of x_4 are represented by $(\text{Pul1}, \text{Pul2}) = (1, 0), (0, 1), (0, 0)$, respectively.

The resulting logistic regression model is

$$\text{logit}(\mu) = \beta_0 + \beta_1 x_1 + \beta_2 x_2 + \beta_3 x_3 + \beta_4 \text{Pul1} + \beta_5 \text{Pul2} + \beta_6 x_5 + \beta_7 (\text{Pul1} \times x_5) + \beta_8 (\text{Pul2} \times x_5).$$

As in Section 4.2, we place the coefficient of the continuous factor β_6 first and the intercept last. We therefore reorder the model parameter vector and assume its nominal value as $\boldsymbol{\theta} = (\beta_6, \beta_1, \beta_2, \beta_3, \beta_4, \beta_5, \beta_7, \beta_8, \beta_0)^\top = (0.35, 1.50, -0.2, -0.15, 0.25, 0.40, 0.10, -0.05, -7.5)^\top$. Then the corresponding predictor vector becomes

$$\mathbf{h}(\mathbf{x}) = (x_5, x_1, x_2, x_3, \text{Pul1}, \text{Pul2}, \text{Pul1} \times x_5, \text{Pul2} \times x_5, 1)^\top.$$

We let $\mathbf{x}_{(1)} = x_5$ denote the continuous factor. Then the derivative used by the continuous search step is

$$\frac{\partial \mathbf{h}(\mathbf{x})}{\partial \mathbf{x}_{(1)}^\top} = (1, 0, 0, 0, 0, 0, \text{Pul1}, \text{Pul2}, 0)^\top.$$

As before, we reorder the factors as $(x_5, x_1, x_2, x_3, x_4)$. Therefore, the range of the continuous factor and the levels of the discrete factors are specified by

```
factor.level = list(c(25,45),c(-1,1),c(-1,1),c(-1,1),c(-1,0,1))
```

We then apply `ForLion_GLM_Optimal()` to obtain a locally D-optimal approximate design under this model. The corresponding R commands and results are listed below:

```
## Note: Always put continuous factors ahead of discrete factors
## After reordering the components in x: x = (x5, x1, x2, x3, x4)
## Here x5--Voltage in [25,45], x1--LotA in {-1,1}, x2--LotB in {-1,1},
## x3--ESD in {-1,1}, x4--Pulse in {-1, 0, 1}
## Code x4 (x[5]) using two dummy variables: Pul1, Pul2
## x -> h(x) = (x5, x1, x2, x3, Pul1, Pul2, Pul1*x5, Pul2*x5, 1)
> hfunc.temp.int <- function(x) {
+   Pul1 <- ifelse(x[5] == -1, 1, 0)
+   Pul2 <- ifelse(x[5] == 0, 1, 0)
+   c(x[1:4], Pul1, Pul2, x[1]*Pul1, x[1]*Pul2, 1)}
# Adjust beta vector length accordingly
#(now 9 terms: 4 main + 2 dummies + 2 interactions + intercept)
> beta.value.int <- c(0.35, 1.50, -0.2, -0.15, 0.25, 0.40, 0.10, -0.05, -7.5)
> variable_names.int <- c("Vol.", "LotA", "LotB", "ESD", "Pul.")
## hprime: derivative wrt x (dh(x)/d(x))
> hprime.temp.int <- function(x) {
+   Pul1 <- ifelse(x[5] == -1, 1, 0)
+   Pul2 <- ifelse(x[5] == 0, 1, 0)
```

```

+           matrix(
+             data = c(
+               1, # derivative wrt Vol.
+               0, # LotA
+               0, # LotB
+               0, # ESD
+               0, # Pul1
+               0, # Pul2
+               Pul1, # interaction Vol*Pul1 wrt Vol
+               Pul2, # interaction Vol*Pul2 wrt Vol
+               0 # intercept
+             ),
+             nrow = 9, ncol = 1, byrow = TRUE)}
> set.seed(482)
> forlion_GLM_int <- ForLion_GLM_Optimal(n.factor = c(0, 2, 2, 2, 3),
+   factor.level = list(c(25,45),c(-1,1),c(-1,1),c(-1,1),
+   c(-1,0,1)), var_names = variable_names.int,
+   hfunc = hfunc.temp.int, h.prime = hprime.temp.int,
+   bvec = beta.value.int, link = "logit", delta0 = 0.01,
+   epsilon = 1e-8, reltol = 1e-4, random = TRUE,
+   nram = 1, delta = 0.08, maxit = 1200, logscale = TRUE)

```

```
> print(forlion_GLM_int)
```

```
Design Output
```

```
=====
```

Count	Vol.	LotA	LotB	ESD	Pul.	Allocation
1	25.0000	1.0000	-1.0000	1.0000	0.0000	0.0308
2	25.0000	1.0000	1.0000	-1.0000	0.0000	0.0386
3	25.0000	-1.0000	-1.0000	-1.0000	1.0000	0.0869
4	25.0000	-1.0000	-1.0000	1.0000	1.0000	0.0141
5	25.0000	-1.0000	1.0000	-1.0000	1.0000	0.0134
6	32.8659	-1.0000	-1.0000	1.0000	0.0000	0.0490
7	34.3536	-1.0000	1.0000	1.0000	0.0000	0.0480
8	25.0000	1.0000	1.0000	1.0000	0.0000	0.0635
9	30.7937	-1.0000	-1.0000	1.0000	1.0000	0.0191
10	25.0000	-1.0000	1.0000	1.0000	1.0000	0.0870
11	31.1887	-1.0000	1.0000	-1.0000	1.0000	0.0280
12	25.0000	-1.0000	-1.0000	1.0000	0.0000	0.0673
13	33.2984	-1.0000	1.0000	-1.0000	0.0000	0.0527
14	32.2390	-1.0000	1.0000	1.0000	1.0000	0.0787
15	25.0000	-1.0000	-1.0000	-1.0000	0.0000	0.0304
16	25.0000	-1.0000	1.0000	-1.0000	-1.0000	0.0110
17	29.6334	-1.0000	1.0000	1.0000	-1.0000	0.1111
18	25.0014	-1.0000	1.0000	1.0000	-1.0000	0.1027
19	25.0602	-1.0000	1.0000	-1.0000	0.0000	0.0679

```
=====
```

```
m:
```

```
[1] 19
```

```
det:
```

```
[1] 6.404087e-10
```

```
convergence:
```

```
[1] TRUE
```

```
min.diff:
```

```
[1] 1
```

```
x.close:
```

```
      [,1] [,2] [,3] [,4] [,5]
[1,]   25  -1  -1  -1   1
[2,]   25  -1  -1  -1   0
itmax:
[1] 1160
```